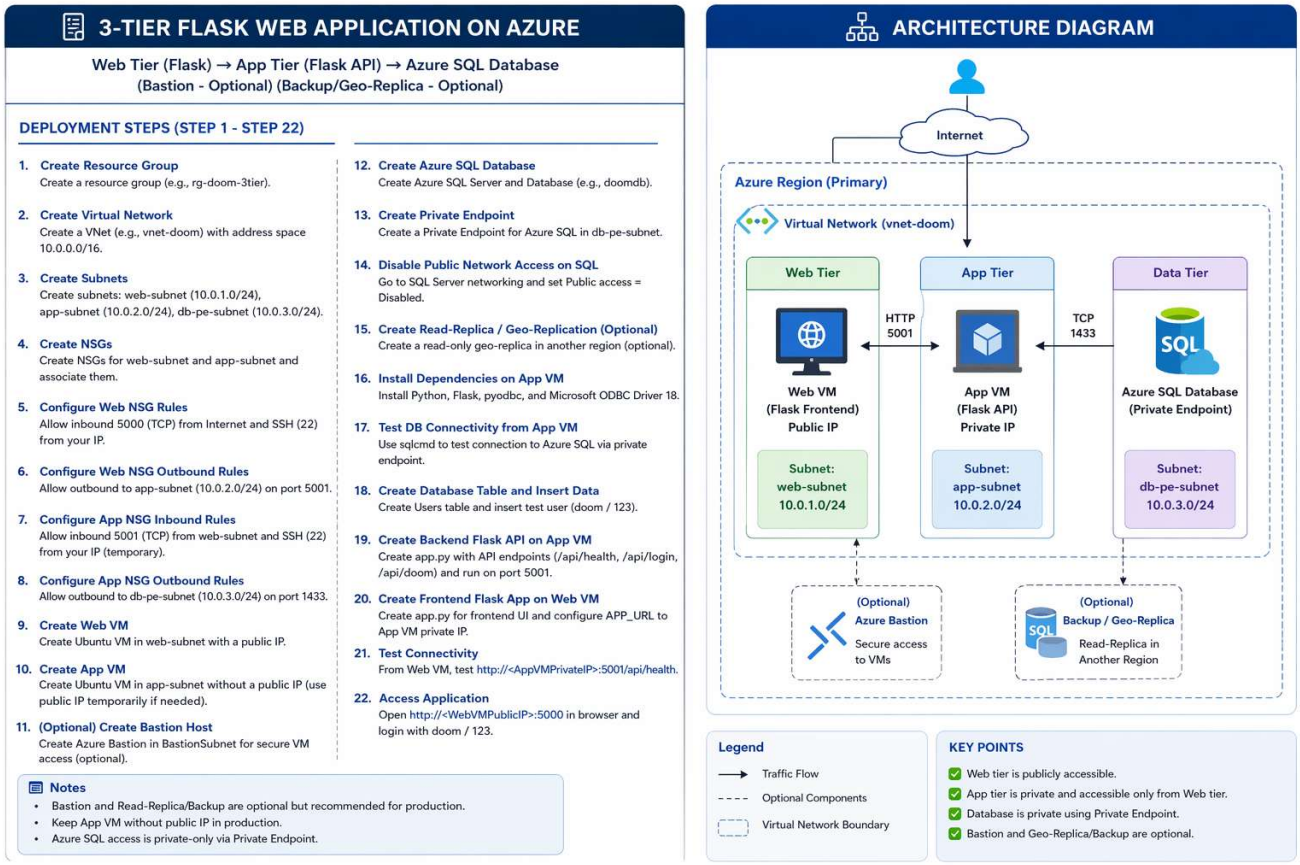
3-Tier Flask Web Application on Azure

A production-style deployment of a Flask-based web application using a dedicated Web tier, App tier, and Azure SQL Database secured with a Private Endpoint.

Architecture	Web VM -> App VM -> Azure SQL Database
Frontend	Python Flask on Web VM
Backend	Python Flask API on App VM
Database	Azure SQL Database with Private Endpoint
Optional Components	Azure Bastion, Backup, Geo-Replica

Architecture Diagram

The architecture below represents the final deployed solution. The Web VM exposes the frontend to end users, the App VM processes API requests privately, and Azure SQL Database serves as the managed data tier. Azure Bastion and backup/geo-replication are shown as optional enhancements.



Final project architecture used in this deployment.

Detailed Deployment Guide - Step 1 to Step 22

The following steps document the exact deployment sequence followed for the project. Each step is written in a portfolio-ready format so the document can be uploaded to GitHub as supporting project documentation.

Step 1 - Create the Resource Group

Create a dedicated Resource Group to logically organize all resources for the 3-tier project. This provides a clean administrative boundary for compute, networking, and database resources.

Step 2 - Create the Virtual Network

Create a Virtual Network named vnet-doom. This VNet acts as the private network boundary for all tiers and allows secure internal communication between the virtual machines and Azure SQL private endpoint.

Step 3 - Create the Subnets

Create three subnets inside the VNet: web-subnet for the frontend VM, app-subnet for the backend VM, and private-end-subnet for the Azure SQL private endpoint. This segmentation enforces tier isolation and follows a real-world network design.

Step 4 - Create Network Security Groups

Create NSGs for the Web and App tiers. NSGs are used to control inbound and outbound traffic so that only required ports and sources are permitted.

Step 5 - Configure Web Tier Inbound Rules

Allow TCP port 5000 from the Internet to expose the frontend Flask application. Allow TCP port 22 only from your admin public IP to manage the Web VM securely.

Step 6 - Configure Web Tier Outbound Rules

Allow the Web VM to communicate with the App VM on TCP port 5001. This enables the frontend to forward login requests and application calls to the backend API.

Step 7 - Configure App Tier Inbound Rules

Allow TCP port 5001 only from the Web tier private address space so the backend is reachable only from the frontend tier. This prevents direct public access to the API.

Step 8 - Configure App Tier Outbound Rules

Allow the App VM to reach Azure SQL Database over TCP port 1433 through the private endpoint subnet. This keeps database connectivity private and prevents public database exposure.

Step 9 - Create the Web VM

Deploy an Ubuntu VM in the web-subnet with a public IP. This VM hosts the Flask frontend and is the only publicly accessible compute layer in the application.

Step 10 - Create the App VM

Deploy an Ubuntu VM in the app-subnet. This VM hosts the Flask backend API and communicates privately with both the Web tier and the database tier.

Step 11 - Install Python and Base Dependencies

Install Python 3, pip, virtual environment packages, and required Linux dependencies on both virtual machines. These packages are required to run Flask applications in a clean and isolated environment.

Step 12 - Create Azure SQL Database

Deploy Azure SQL Database as the data tier. This provides a managed relational database service without the need to maintain a database VM.

Step 13 - Configure the Private Endpoint

Create a private endpoint for Azure SQL inside the private-end-subnet. This maps Azure SQL to a private IP address within the VNet, allowing the App VM to connect securely over private networking.

Step 14 - Disable Public Access on Azure SQL

Disable public network access on the SQL server after confirming the private endpoint is working. This ensures database traffic remains private and aligns the project with production-style security practices.

Step 15 - Create the Database Table and Seed Data

Using sqlcmd from the App VM, create the Users table and insert a test user record. This validates connectivity from the application tier to the managed database tier.

Step 16 - Test Database Connectivity from the App VM

Use sqlcmd to connect to the Azure SQL database through the private endpoint and confirm the database is reachable privately. This verifies the network design before deploying application code.

Step 17 - Create the Backend Flask API

Create the backend Flask application on the App VM. Expose endpoints such as /api/health, /api/login, and /api/doom to process application requests and return structured responses.

Step 18 - Run the Backend on Port 5001

Start the backend application on TCP port 5001 and bind it to 0.0.0.0 so it is reachable from the Web VM across the private network.

Step 19 - Test the Backend API

Run curl commands from the App VM to validate /api/health and /api/doom. This confirms that the backend service is functioning correctly before integrating the frontend.

Step 20 - Create the Frontend Flask Application

Create the frontend Flask app on the Web VM. This application presents the login page to the user and forwards login requests to the backend API on the App VM.

Step 21 - Test Web-to-App Connectivity

From the Web VM, test connectivity to the App VM using curl against the backend API. A successful response confirms the frontend and backend tiers can communicate over the private network.

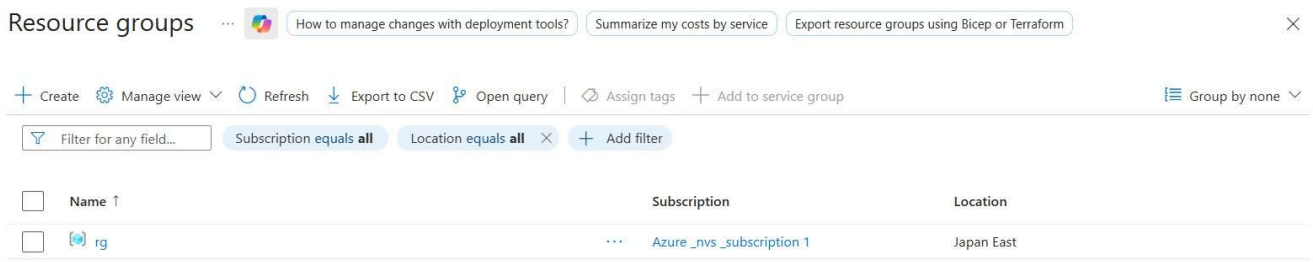
Step 22 - Validate the Application in the Browser

Open the Web VM public IP on port 5000, log in with the test credentials, and verify the Doctor Doom page loads successfully. This confirms the full end-to-end flow across Web, App, and Data tiers.

Implementation Evidence and Validation Screenshots

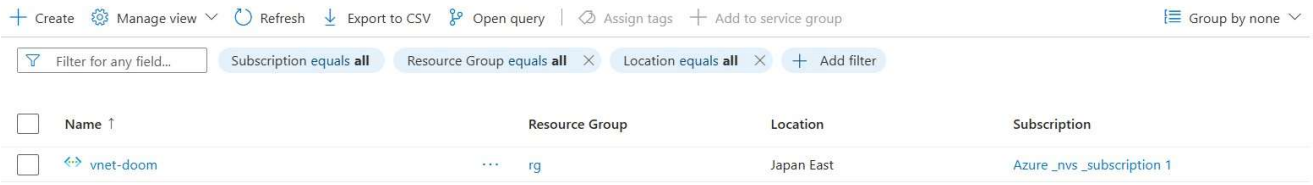
The following screenshots capture the actual deployment progress and successful validation of the solution. Images are resized to fit the page cleanly and paired with professional captions.

Figure 1. Resource Group created for the project deployment.



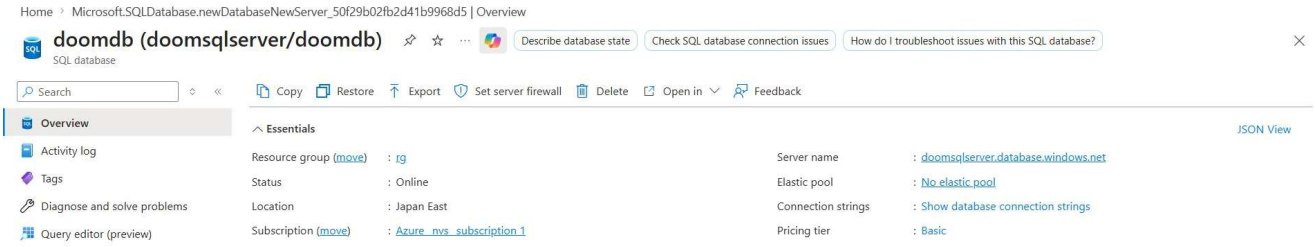
Rg 1

Figure 2. Virtual Network created to host web, app, and private endpoint subnets.



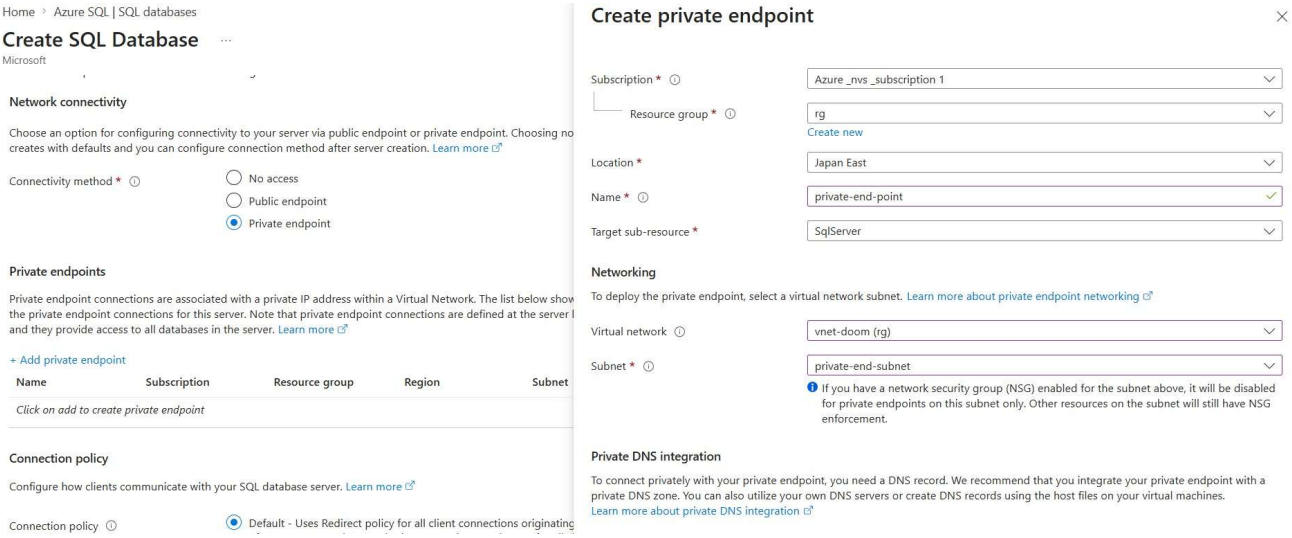
Vnet Doom

Figure 3. Azure SQL Database created as the data tier.



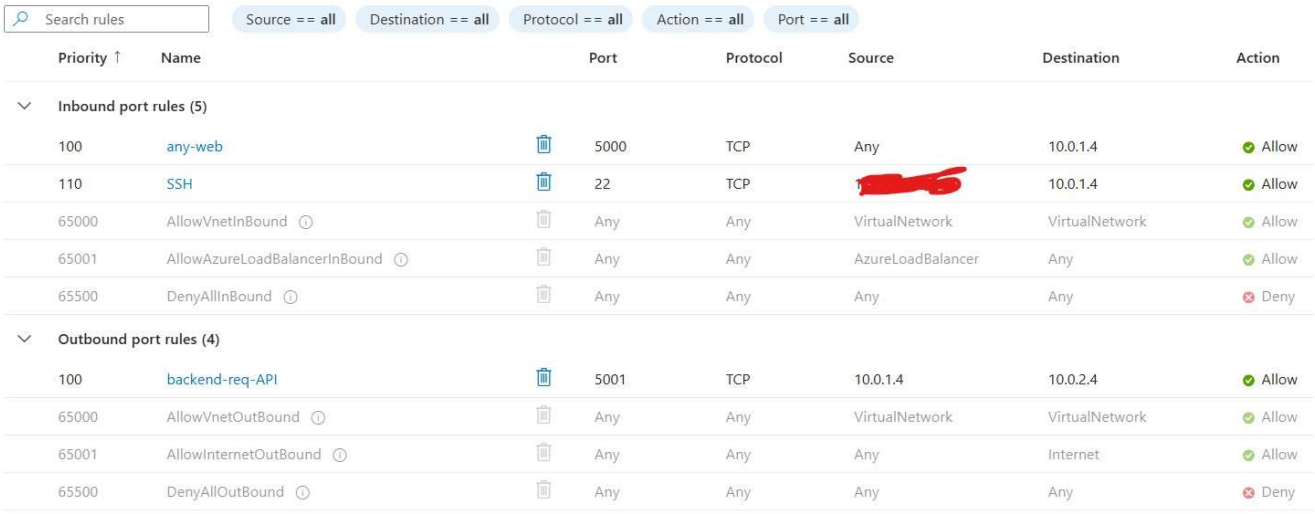
Database Created

Figure 4. Private Endpoint configured for Azure SQL Database.



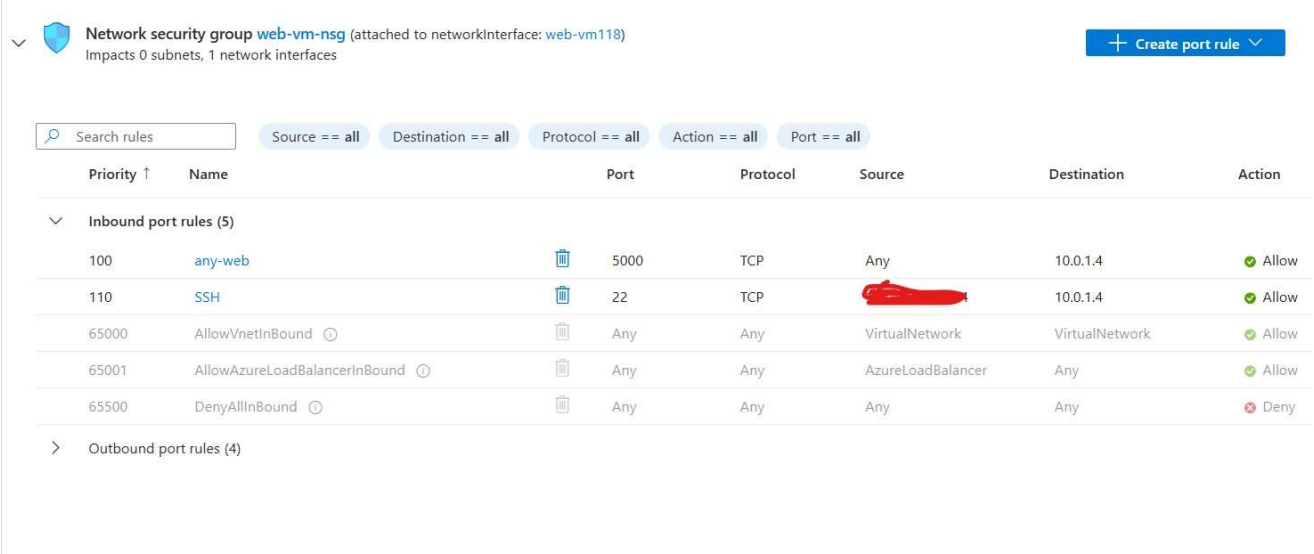
Sqldatabase Private Endpoint Creation

Figure 5. Web tier NSG rules allowing website access on port 5000 and controlled SSH access.



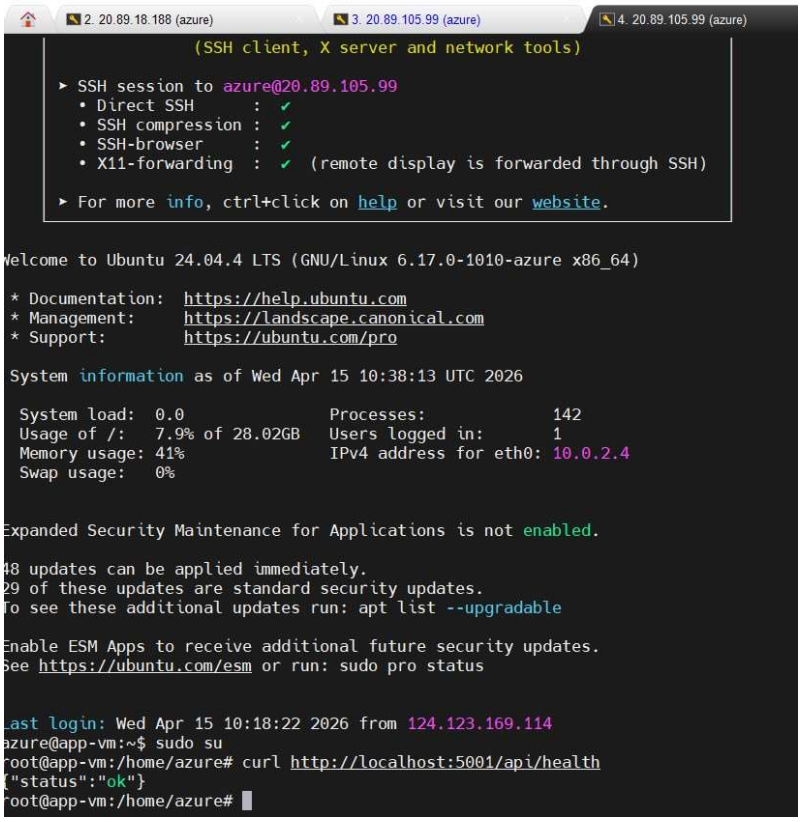
Web Nsg Rules

Figure 6. App tier NSG rules allowing backend API communication from the web tier.



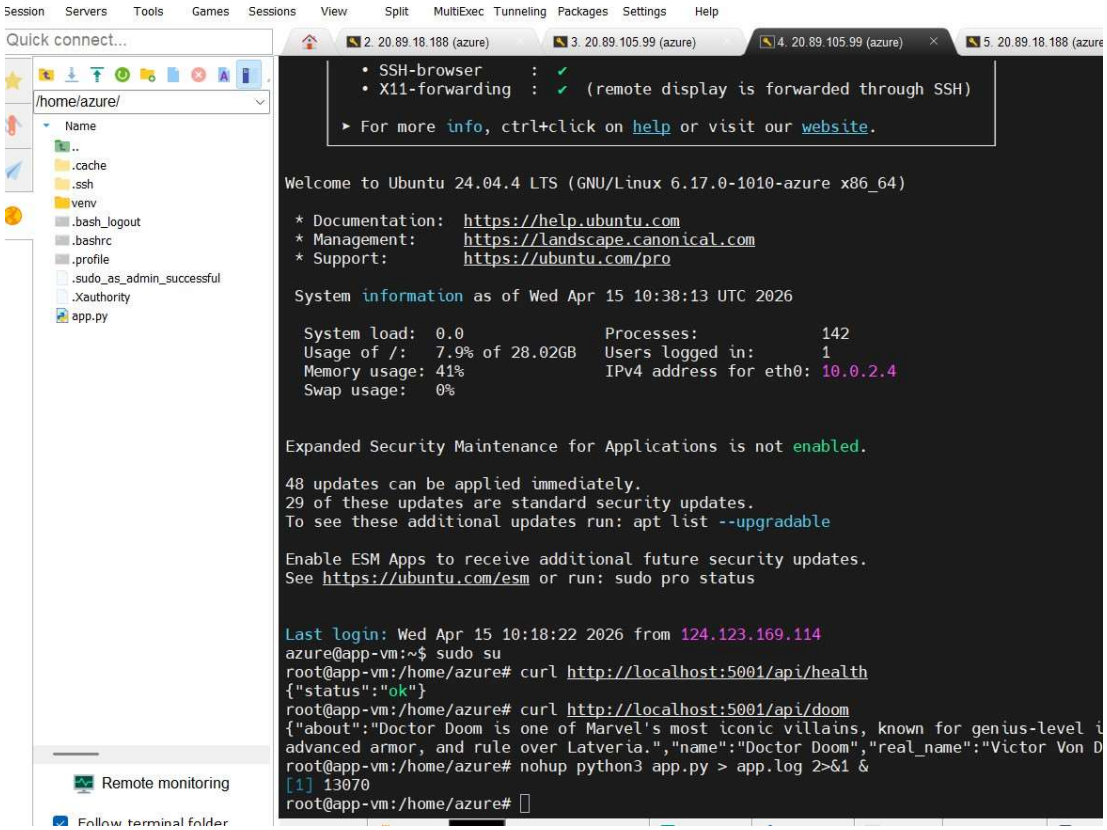
App Nsg Rules

Figure 7. Backend Flask API validated successfully on the App VM.



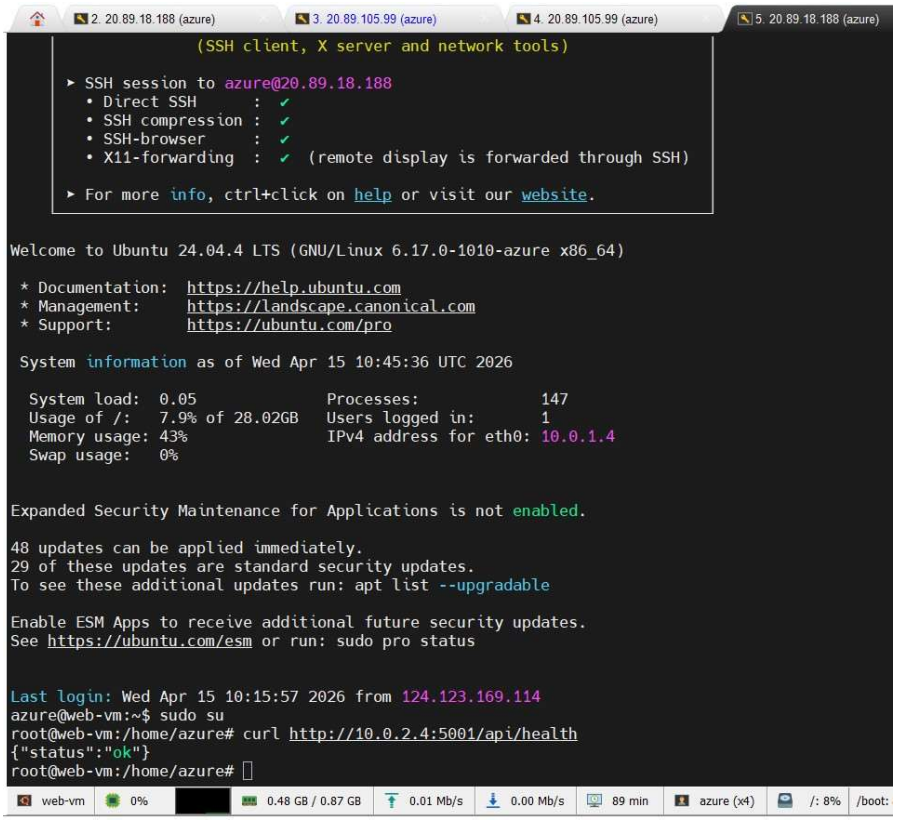
Open In New App Terminal To Test, Backend Falsk Is Running Good

Figure 8. Backend application running in background on App VM.



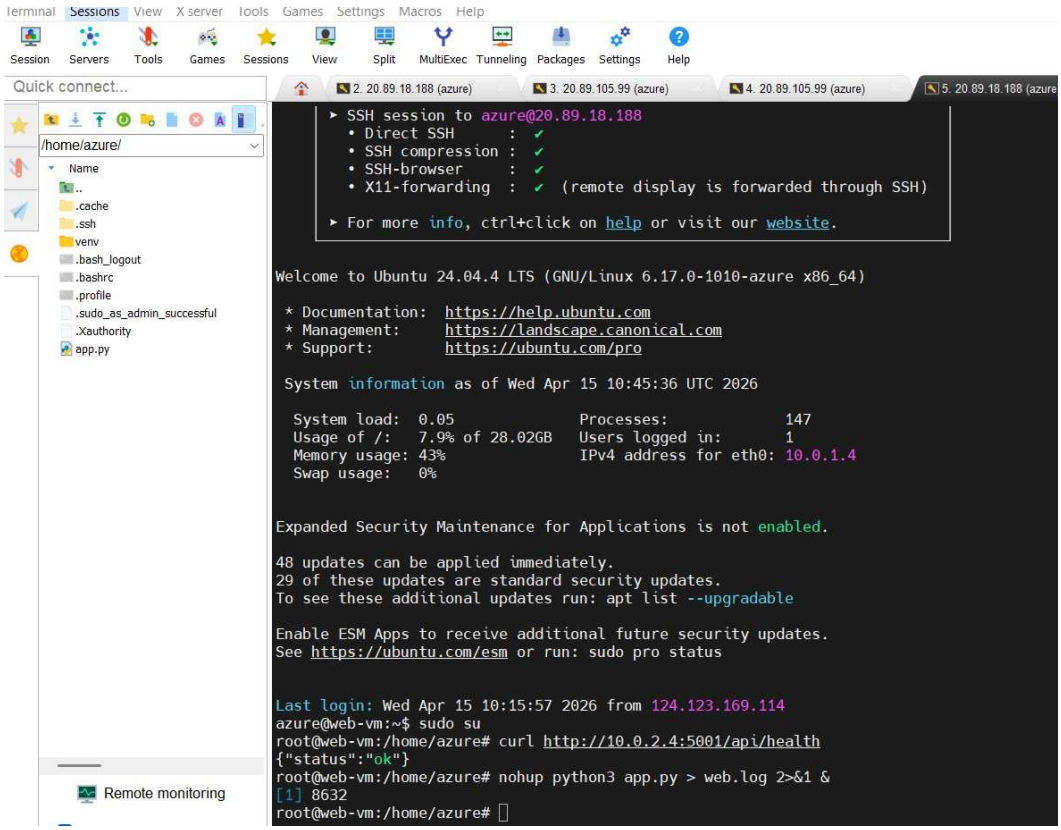
Apps Runs In Background Appvm

Figure 9. Web VM successfully communicating with App VM over the private network.



Test Web Ap Vm Connectivity Frontend Backend App Flsk Api Communication

Figure 10. Frontend application running in background on Web VM.

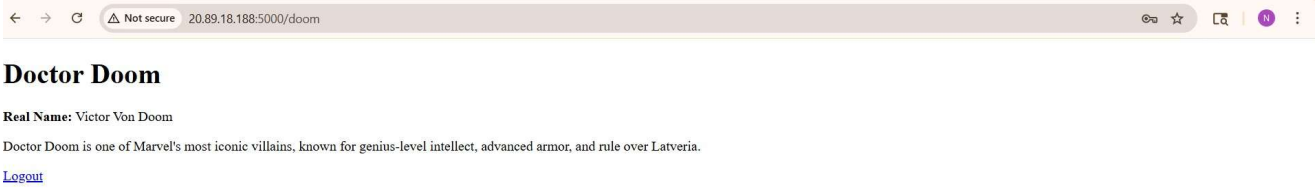


Apps Runs In Background Webvm

Figure 11. Login page exposed from the web tier.

Website Login

Figure 12. Successful login displaying the Doctor Doom application page.



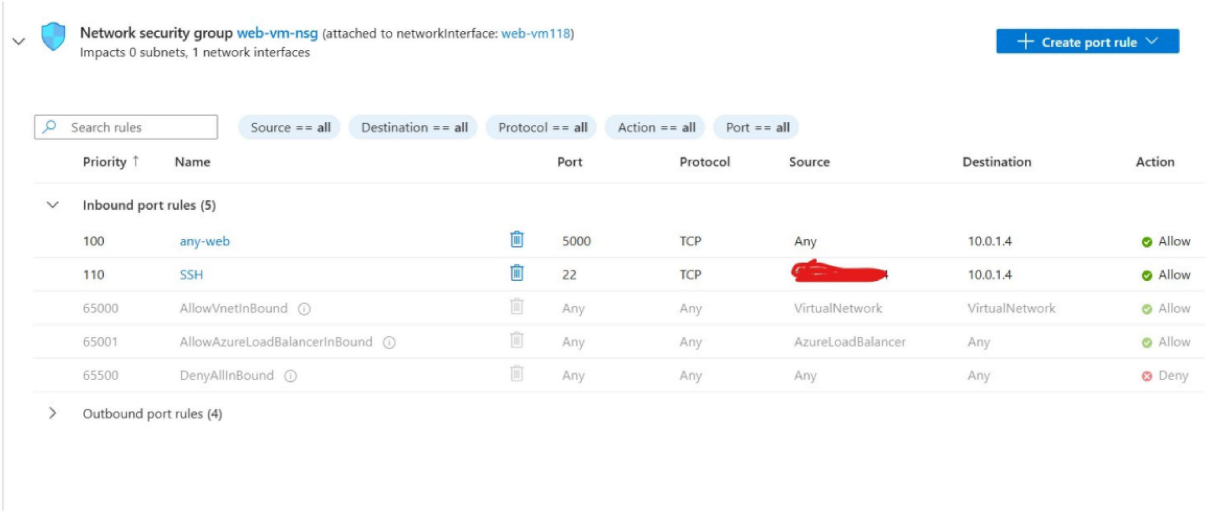
Website Working After Login

3-Tier Flask Application on Azure (Final Corrected Version)

Updated Correction (Figure 6): App NSG rules are corrected to allow port 5001 (Web → App communication) with destination as App VM private IP (10.0.2.4). SSH rule also correctly targets App VM.

Figure 6: Corrected App NSG Rules

Figure 6. App tier NSG rules allowing backend API communication from the web tier.



App Nsg Rules

Explanation: App NSG must allow inbound traffic only from Web Tier on port 5001. This ensures backend API is accessible only from frontend and not publicly. SSH should be restricted to admin IP and target App VM private IP.